

Otimização da Distribuição de Disciplinas a Professores Baseada em Busca Local

Carolina Noda Morishita

Faculdade de Tecnologia (FT) - Universidade Estadual de Campinas (UNICAMP)
R. Paschoal Marmo, 1888 - Limeira, SP
c254187@dac.unicamp.br

Luis A. A. Meira

Faculdade de Tecnologia (FT) - Universidade Estadual de Campinas (UNICAMP)
R. Paschoal Marmo, 1888 - Limeira, SP
meiraft@unicamp.br

RESUMO

O problema de atribuir professores a disciplinas é antigo e encontrado em todas as instituições de ensino ao redor do mundo. Apesar disso, muitas instituições de ensino resolvem o problema de forma não automática. Este artigo desenvolve um programa que realiza a atribuição de professores a disciplinas de maneira automática. O problema foi modelado como uma minimização biobjetivo, onde um objetivo consiste em atender a aptidão do professor em ministrar determinada disciplina e o segundo foi realizar um balanceamento de carga. A otimização foi feita por meio de um algoritmo de busca local e os dois objetivos foram unidos por uma soma ponderada. O resultado do algoritmo foi satisfatório, tendo mais qualidade que a solução utilizada na prática. O resultado foi promissor, podendo ser usado na prática pelo tomador de decisão. Entretanto o problema real é mais complexo, envolvendo uma gama de critérios que não foram abordados neste trabalho.

PALAVRAS CHAVE. Problema de atribuição, Otimização, Busca Local.

MH - Metaheurísticas, OD - Otimização Discreta

ABSTRACT

The problem of assigning teachers to courses is longstanding and encountered in educational institutions worldwide. Despite this, many institutions still solve the problem manually. This paper develops a program that performs the assignment of teachers to courses automatically. The problem was modeled as a bi-objective minimization, where one objective is to match the teacher's aptitude for teaching a given course, and the second is to balance the workload. Optimization was performed using a local search algorithm, and the two objectives were combined using a weighted sum. The result of the algorithm was satisfactory, showing higher quality than the solution currently used in practice. The outcome was promising and could be applied in real decision-making scenarios. However, the real-world problem is more complex, involving a range of criteria that were not addressed in this work.

KEYWORDS. Assignment problem, Optimization, Local Search.

MH - Metaheuristics, OD - Discrete Optimization

1. Introdução

O problema definido como *timetabling* não é novo e diversos pesquisadores já introduziram modelos e algoritmos para formulá-lo e resolvê-lo [MirHassani, 2006]. O processo tem como objetivo organizar um determinado número de eventos para uma série de recursos e garantir que um conjunto predefinido de restrições seja satisfeito [Song et al., 2018]. Esse tema tem sido frequentemente estudado devido à sua ampla gama de aplicações e, nesse trabalho trataremos a aplicação na alocação de disciplinas para professores, mais especificamente dos cursos de Tecnologia em Análise e Desenvolvimento de Sistemas (TADS) e Bacharelado em Sistemas de Informação (BSI) da Unicamp.

A atribuição de professores universitários para ministrar as disciplinas pode ser um processo complexo e demorado a ser realizado semestralmente pela coordenação acadêmica. Esse desafio decorre da necessidade de conciliar múltiplos fatores, como os interesses individuais dos docentes em relação às disciplinas e a garantia de uma distribuição equilibrada da carga horária entre os professores. O elevado número de disciplinas e docentes envolvidos pode resultar em diversas combinações possíveis de atribuições, o que explica a complexidade desse problema. Diante desse cenário, torna-se evidente a necessidade de uma solução que não apenas simplifique essa tarefa, como também aumente a eficiência e precisão na alocação dos docentes.

Ao longo da última década, houve um grande interesse na aplicação de algoritmos baseados em metaheurísticas a problemas de *timetabling* de universidades, talvez devido ao fato de que esses problemas podem variar bastante entre as instituições e que as metaheurísticas são, por definição, tipos de algoritmos de "nível superior" de propósito geral que podem ser usados com uma ampla gama de diferentes tipos de problemas [Lewis, 2008].

Nos últimos 4 anos, o Simpósio Brasileiro de Pesquisa Operacional (SBPO) publicou nove trabalhos relacionados a alocação de disciplinas. Um dos trabalhos foi de Garruth e Resendo [2024], que utilizaram Programação Linear Inteira para resolver o problema de agendamento de horários escolares de institutos federais de ensino. Os autores apresentaram o conceito de custo pedagógico, onde há penalizações para conflitos de horários e também consideraram as preferências dos docentes.

Outro trabalho do mesmo ano foi de Oliveira et al. [2024], que também utilizou Programação Linear Inteira, a linguagem Python e o solver IBM ILOG CPLEX Optimization Studio para auxiliar na alocação de horários do Instituto Federal de Minas Gerais (IFMG) - Campus Congonhas. O software comercial IBM ILOG CPLEX Optimization Studio é uma ferramenta baseada em programação inteira e foi utilizada apenas para gerar uma versão inicial dos horários, pois o software foi incapaz de alocar as aulas respeitando as restrições fortes, que são restrições que devem ser rigorosamente atendidas para que haja uma solução viável.

Em 2023, o TCC de Mariotti e Filho [2023] propôs o desenvolvimento de uma ferramenta que substituisse um software terceirizado e pago que auxilia o desenvolvimento de quadros de horários do Instituto Federal Goiano - Campus Rio Verde, mas que possui poucas possibilidades de alterações de critérios. O autor desenvolveu a ferramenta por meio de uma heurística construtiva, que primeiramente gera horários factíveis e depois é feita melhorias utilizando o método *fix-and-optimize*.

Em 2022, o trabalho de Kampke e Müller [2022] analisou os impactos do fator de aleatoriedade e de movimentos na meta-heurística híbrida *Greedy Randomized Adaptive Search Procedure (GRASP)* com *Simulated Annealing (SA)*, e os autores implementaram o movimento *Lecture Move* para a resolução do Problema de Tabela-Horário do Departamento de Computação (DCOMP) do Centro de Ciências Exatas, Naturais e da Saúde (CCENS) da Universidade Federal do Espírito Santo (UFES). Segundo Kampke e Müller [2022], o movimento *Lecture Move* consiste na troca

entre duas ofertas ou entre uma oferta e um horário vazio. Ou seja, o *Lecture Move* corresponde a uma combinação de Move e Swap, em que são escolhidos aleatoriamente dois horários na tabela-horário. Caso apenas um deles esteja vazio é realizado um Move, mas se ambos possuírem ofertas alocadas é realizado um Swap.

Objetivo Principal: Esta pesquisa, baseado em um problema real encontrado pela coordenação dos cursos de informática da Faculdade de Tecnologia da UNICAMP, teve como foco buscar uma abordagem metaheurística para a otimização do processo de atribuição de professores a disciplinas permitindo que o trabalho da coordenação seja facilitado.

Etapas para se chegar ao objetivo:

- Interpretação da planilha de aptidões.
- Definição de duas funções objetivo.
- Definição dos pesos de atribuição.
- criação da solução inicial
- Ajuste manual de parâmetros para obter soma ponderada das funções objetivo.
- Implementação de um algoritmo de busca local.
- Otimização
- Análise dos resultados.

2. Fundamentos

Talbi [2009] definiu o processo de tomada de decisão de modelos de otimização em quatro passos: formulação do problema, modelagem do problema, otimização do problema e implementação da solução. Na formulação, um problema é identificado e uma declaração inicial é feita. Na modelagem, um modelo matemático abstrato é construído para o problema. Uma vez que o problema é modelado, o procedimento de resolução gera uma solução “boa” para o problema. A solução obtida é testada praticamente pelo tomador de decisão e é implementada se for “aceitável”. Algum conhecimento prático pode ser introduzido na solução a ser implementada. Se a solução for inaceitável, o modelo e/ou o algoritmo de otimização tem que ser melhorado e o processo de tomada de decisão é repetido.

Segundo Kochenderfer e Wheeler [2019], com o desenvolvimento do computador eletrônico, o interesse no processo de otimização se intensificou. A facilidade e rapidez da máquina em realizar cálculos permitiu que a otimização fosse aplicada a problemas muito maiores em uma variedade de domínios, estimulando o interesse na criação de algoritmos numéricos para otimização. Um dos maiores avanços veio com a introdução da programação linear, que é um problema de otimização com uma função objetivo linear e restrições lineares.

Nos últimos anos, as técnicas de otimização avançaram rapidamente e um progresso considerável foi alcançado. Ao mesmo tempo, os computadores digitais se tornaram mais rápidos, mais versáteis e mais eficientes. Como consequência, agora é possível resolver problemas complexos de otimização que eram considerados intratáveis há apenas alguns anos [Kochenderfer e Wheeler, 2019].

2.1. Otimização multi-objetivo

Segundo Gunantara [2018], a otimização multi-objetivo refere-se a encontrar os valores de soluções ótimas de mais de um objetivo desejado. A otimização multi-objetivo busca um conjunto de soluções que representam diferentes compromissos entre os objetivos (trade-offs). Nesse tipo de otimização, não há uma única melhor solução para todos os propósitos, mas sim várias soluções. O economista e sociólogo Vilfredo Pareto introduziu o conceito de eficiência, também conhecido como Ótimo de Pareto, propondo que, em um sistema com múltiplos objetivos, uma solução é “ótima” quando não é possível melhorar um objetivo sem piorar outro [Kochenderfer e Wheeler, 2019]. A importância fundamental da eficiência (ótimo de Pareto) é baseada na observação de que qualquer x que não seja eficiente não pode representar uma alternativa mais preferida para um tomador de decisão, porque existe pelo menos uma outra solução viável $x' \in X$ tal que $f_k(x') \leq f_k(x)$ para todo $k = 1, \dots, p$, onde a desigualdade estrita se mantém pelo menos uma vez, ou seja, x' deve ser claramente preferido a x [Ehrgott, 2005].

2.2. Metaheurística

De acordo com Talbi [2009], a metaheurística pode fornecer soluções “aceitáveis” em um tempo razoável de problemas difíceis e complexos ao explorar o espaço de busca das soluções, reduzindo o tamanho efetivo do espaço ou explorando-o de forma eficiente. Em geral, as metaheurísticas possuem três propósitos principais: resolver problemas mais rapidamente, resolver grandes problemas e obter algoritmos robustos. Além disso, são simples de projetar e implementar, e são muito flexíveis. Para identificar a necessidade de aplicabilidade desta abordagem, deve-se considerar a complexidade do problema, o tamanho das instâncias de input que o algoritmo deve resolver e o tempo de busca necessário para resolver um determinado problema.

2.3. Vizinhança e Métodos de Solução Única

Talbi [2009] definiu os métodos de solução única como métodos que possuem iterações de duas fases, a de geração e de substituição da solução única atual. Na fase de geração, um conjunto de soluções candidatas é gerado a partir das transformações da solução atual s , onde o conjunto $C(s)$ é geralmente obtido por transformações locais da solução. Na fase de substituição, uma seleção é realizada do conjunto de soluções candidatas $C(s)$ para substituir a solução atual; ou seja, uma solução $s' \in C(s)$ é selecionada para ser a nova solução. A vizinhança pode ser definida como: para qualquer solução s de S , um conjunto $N(s) \subset S$ que é um conjunto de soluções vizinhas s [Dreo et al., 2006].

As metaheurísticas baseadas em solução única trabalham com uma única solução por vez, iterativamente melhorando-a ao explorar sua vizinhança. Conforme descrito por Luke [2013], para otimizar uma solução utilizando-se deste método, é necessário seguir pelo menos quatro etapas fundamentais:

- Fornecer uma ou mais soluções candidatas iniciais. Isso é conhecido como procedimento de inicialização.
- Avaliar a qualidade de uma solução candidata. Isso é conhecido como procedimento de avaliação.
- Fazer uma cópia de uma solução candidata.
- Ajustar uma solução candidata, o que produz uma solução candidata ligeiramente diferente aleatoriamente. Isso, mais a operação de cópia, são conhecidos coletivamente como procedimento de modificação.

O algoritmo normalmente fornecerá um procedimento de seleção que decide quais soluções candidatas manter e quais rejeitar à medida que explora o espaço de soluções possíveis para o problema.

2.4. Busca local

A busca local é um dos exemplos mais simples e antigos dentro da categoria de métodos single-solution [Talbi, 2009]. Segundo o autor, o método de busca local, a cada iteração, substitui a solução atual por uma vizinha que melhora a função objetivo. A busca para quando todos os vizinhos candidatos são piores do que a solução atual, significando que um ótimo local é alcançado.

3. Metodologia

Esta seção está dividida em suas subseções: Subseção 3.1 que contém a modelagem do problema e Subseção 3.2 que contém detalhes da implementação da solução.

3.1. Modelagem do problema

A alocação de disciplinas para professores pode ser modelada como um problema de otimização combinatória, cujo objetivo é distribuir de maneira eficiente as disciplinas entre os professores, considerando um conjunto de requisitos. Esse problema é comum em instituições de ensino, como escolas e universidades, e envolve a definição de qual professor leciona qual disciplina. O problema consiste em maximizar a eficiência e a satisfação de docentes, alunos e da administração escolar. Durante esse processo, foram levantados dois requisitos principais: garantir uma carga horária balanceada entre os docentes e considerar a aptidão de cada professor para ministrar determinadas disciplinas.

Primeiramente, foi criada uma planilha pela coordenação para o controle de aptidões de cada professor. Veja a Figura 1. A primeira coluna é o identificador (ID) de cada disciplina, podendo utilizar números inteiros ou siglas. A segunda coluna possui o nome das disciplinas. A terceira coluna é o número de créditos de cada disciplina e a quarta coluna contém os oferecimentos das disciplinas (se serão oferecidas para BSI, TADS ou Serviço). A informação relevante a coluna *Oferecimentos* é que uma disciplina precisa ser oferecida várias vezes. Por exemplo, uma vez para TADS, uma vez para BSI e uma vez como serviço. Estes valores estão dentro de uma célula separados por vírgula.

	A	B	C	D	E	F	G	H
1	ID	Disciplinas	Créditos em Sala	Oferecimentos	Ana	Beatriz	Carlos	Daniela
2	TT001	Administração d	4	Serviço	-1	-1	-1	-1
3	TT002	Administração d	4	Serviço	-1	-1	-1	-1
4	TT003	Algoritmos e Pro	6	BSI,TADS	-1	-1	-1	0
5	TT004	Algoritmos e Pro	6	BSI,TADS	-1	-1	-1	3
6	TT005	Análise de Sister	4	Serviço	-1	-1	-1	-1
7	TT006	Análise de Sister	2	Serviço	-1	-1	-1	-1
8	TT007	Atividades prátic	2	Serviço	-1	-1	0	-1
9	TT008	Banco de Dados	4	Serviço	-1	-1	0	4
10	TT009	Banco de Dados	2	TADS	-1	-1	4	4
11	TT010	Cálculo I	4	BSI, TADS, Serv	-1	-1	4	3
12	TT011	Cálculo Numéric	2	BSI	1	-1	-1	0
13	TT012	Ciência, Tecnolo	2	BSI,TADS	5	4	1	4

Figura 1: Exemplo da planilha csv utilizada no programa.

A partir da quinta coluna, cada coluna representa um professor nas quais suas células indicam suas respectivas aptidões para ministrar determinadas disciplinas, com valores possíveis

de -1 a 5. Se o professor possui interesse em ministrar a disciplina, é colocado um número de 1 a 5, onde 1 representa menos aptidão e 5 representa aptidão máxima. O valor zero indica que o professor não se sente totalmente capacitado para ministrar a disciplina, mas, caso seja designado para essa função, não há problemas em aceitá-la. Por outro lado, o valor -1 representa desinteresse, significando que o professor não tem aptidão para lecionar a disciplina em questão.

Existem duas versões desta planilha, uma com as disciplinas que serão oferecidas em um dado semestre e outra com as disciplinas que serão oferecidas em uma ano, ou seja, dois semestres. Isso é transparente para o programa, que irá funcionar de maneira semelhante nos dois casos.

Nesta pesquisa trabalhamos com uma versão da planilha de aptidões cujos dados foram tornados anônimos. Note que nesta planilha, os nomes dos professores foram substituídos para um nome genérico, como Ana, Beatriz, Carlos, etc. Para garantir que as informações sejam anônimas, foi feita uma permutação arbitrária nos nomes das disciplinas. Por esta razão, na Figura 1 temos Cálculo com 4 créditos e Algoritmos e Prog. com 6 créditos. O programa será construído nesta planilha anonimizada e depois será executado utilizando os dados reais.

3.2. Implementação

O projeto procurou criar uma solução utilizando-se do método de busca local, seguindo os passos que categorizam esse método. O desenvolvimento do código foi realizado utilizando a linguagem Python, com o suporte de bibliotecas como pandas, para a seleção e manipulação de dados provenientes de planilhas, matplotlib, para a construção de gráficos e random, que trabalha com valores aleatórios.

Como uma disciplina pode ser oferecida mais de uma vez, foi criado o conceito de oferecimento que é um par composto pela disciplina e por um inteiro $\{0, 1, 2 \dots\}$. Isso permite que uma disciplina apareça várias vezes, uma para cada inteiro. A este par é dado o nome de oferecimento.

3.2.1. Criação da Solução Inicial

O algoritmo de busca local para iniciar precisa de uma solução inicial. Nesse método, os oferecimentos são aleatoriamente atribuídos a professores com aptidões maiores que -1, sendo -1 um indicativo de inadequação para ministrar a disciplina. Pelo Algoritmo 1, é possível notar que é iterado a lista de oferecimentos e é criado uma lista para cada oferecimento para armazenar os professores com aptidão maior que -1. Depois, é percorrido todas as colunas dos professores e é verificado a aptidão do professor na disciplina. Se for maior que -1, é adicionado à lista aptos. Por último, é selecionado um professor da lista de aptos de forma aleatória e é atribuído ao oferecimento.

Algoritmo 1: Geração de uma solução inicial aleatória

Entrada: Lista de professores P , lista de oferecimentos D e tabela de aptidões

Saída: Uma solução

para cada $oferecimento \in D$ **faça**

$aptos \leftarrow \emptyset$

para cada $professor \in P$ **faça**

se $aptidao(professor, disciplina) > -1$ **então**

$aptos \leftarrow aptos \cup \{professor\}$

fim

fim

$solucao[oferecimento] \leftarrow escolherAleatorio(aptos)$

fim

3.2.2. Definição das Funções Objetivo

O cálculo do custo-atribuição baseia-se na Tabela 1, que associa os níveis de aptidão dos professores ao custo correspondente. Note que o problema é de minimização, então, quanto menor o custo, melhora solução.

Tabela 1: Relação de atribuição do custo para cada aptidão

Aptidão	Custo
5	1
4	1.3
3	1.5
2	2.0
1	2.5
0	5.0
-1	10.0

Estes custos foram definidos em conjunto com o coordenador associado dos cursos de TADS e BSI. Ele será um dos usuários do sistema, quando pronto. Os pesos foram definidos pensando na prioridade dos requisitos. Por exemplo, devemos priorizar que um professor não seja alocado a uma disciplina que não queira ministrar (-1), então é atribuído um alto custo (valor 10). Os custos das aptidões de 1 a 5 são relativamente baixos, visto que os professores mostraram interesse em ministrar a disciplinas.

Por fim, todos os custos de cada alocação são somados e constituem o custo-atribuição final. O cálculo do custo-atribuição pode ser dado pela Equação 1.

Considere uma variável $x_{ij} = 1$ se o professor $i \in P$ atribuído ao oferecimento $j \in D$, zero caso contrário. Seja c_{ij} o custo de atribuição do professor $i \in P$ para a disciplina $j \in D$.

$$f_{atrib}(S) = \sum_{i \in P} \sum_{j \in D} x_{ij} c_{ij} \quad (1)$$

Para uma dada solução, é necessário calcular o número de créditos de cada professor. Dada uma solução S e um professor $i \in P$, temos que $\rho(i, S)$ é o número de créditos atribuídos a um dado professor.

$$f_{dist}(S) = \sum_{i \in P} \rho(i, S)^2 \quad (2)$$

A função do somatório das cargas didáticas ao quadrado penaliza cargas altas. Por exemplo, 6 créditos divididos entre 3 professores podem ter carga $\{2, 2, 2\}$ com custo $f_{dist} = 2^2 + 2^2 + 2^2 = 12$. Já a distribuição $\{1, 2, 3\}$ terá $f_{dist} = 1^2 + 2^2 + 3^2 = 14$, indicando que é uma solução de qualidade mais baixa. Já a solução $\{1, 1, 4\}$ terá custo $f_{dist} = 18$, sendo a solução com pior custo.

Por fim, o custo combinado é dado pela seguinte equação:

$$f(S) = \frac{0.5}{4047} f_{dist}(S) + \frac{0.5}{152} f_{atrib}(S) \quad (3)$$

A função de custo combinado calcula um valor que representa o custo da solução $f(S)$, combinando dois critérios: a distribuição da carga horária entre os professores e a aptidão dos

professores para as disciplinas que ministram. Ela pondera a penalização da distribuição desigual da carga horária, $f_{dist}(S)$ e o custo da atribuição baseado na aptidão dos professores $f_{atrib}(S)$. Cada critério é normalizado por um fator (4047 e 152, respectivamente) e recebe um peso de 0.5, garantindo uma avaliação equilibrada entre os dois aspectos. Esses fatores foram obtidos após rodar o código e uma solução otimizada para $f_{dist}(S)$ ter custo próximo a 4047 e uma solução otimizada para $f_{dist}(S)$ ter custo próximo a 152.

3.2.3. Classe Otimizador

A classe Otimizador implementa um algoritmo para otimizar a alocação de professores a disciplinas com base em critérios de aptidão e distribuição de carga horária. Recebe como entrada uma tabela de dados contendo informações sobre as disciplinas e a aptidão dos professores para cada uma delas. Possui o método *otimiza*, que realiza a otimização através de um processo iterativo. Nesse método, primeiramente são criadas duas instâncias da classe *Solucao* (*best* e *sol*),

A cada iteração, *sol* é modificado trocando as atribuições de professores utilizando alternadamente *swap* e *swap2*. O *swap* seleciona aleatoriamente uma disciplina e para ela troca seu professor por um professor aleatório. Já *swap2* executa o processo de troca duas vezes.

Se o custo combinado da nova solução for a melhor que a melhor encontrada, ou seja, (*best*), a melhor solução é substituída por *sol*. Caso contrário, *sol* é revertida para *best*. Esse processo se repete até atingir um limite de iterações, definido até 1 milhão, ou até que seja interrompido manualmente.

4. Resultados

Neste primeiro experimento o programa utilizou como entrada uma planilha anual, ou seja, fazendo a atribuição para dois semestres. Ao finalizar a execução do programa manualmente, é exibido no terminal da IDE a alocação das disciplinas, organizada por professor. A Figura 2 exemplifica uma parte do resultado mostrado no terminal, onde nota-se a alocação das disciplinas para as professoras Ana e Beatriz. Para cada professor são mostradas quais disciplinas em específico foram alocadas, junto da aptidão do professor para tal matéria. A carga horária total é calculada e também mostrada no terminal.

```
Professor: Beatriz
Carga Horária: 12
Disciplinas:
TT015 - Desafio Unicap de Empreendedorismo Tecnológico (Aptidão 4)
TT019 - Engenharia de Software II (Aptidão 5)
TT023 - Estruturas de Arquivos (Aptidão 5)
TT035 - Introdução à Programação em Python (Aptidão 4)
TT038 - Lógica Matemática (Aptidão 3)

Professor: Ana
Carga Horária: 12
Disciplinas:
TT012 - Ciência, Tecnologia e Sociedade (Aptidão 5)
TT012 - Ciência, Tecnologia e Sociedade (Aptidão 5)
TT021 - Estrutura de Dados I (Aptidão 5)
TT033 - Introdução a Interfaces Humano-Computador (Aptidão 5)
```

Figura 2: Parte do terminal da IDE após interrupção manual do programa, mostrando os resultados da alocação de disciplinas.

Para uma visualização geral dos resultados, foram gerados gráficos. A Figura 3 mostra o gráfico da carga horária total anual para cada professor. A análise revela que a carga horária total por professor obteve uma distribuição equilibrada, conforme planejado. A carga horária mínima registrada foi de 12 créditos, enquanto a máxima atingiu 16 créditos (considerando que a planilha com disciplinas de dois semestres foi utilizada). Ou seja, houve pouca discrepância entre professores que possuem a menor carga total e professores que possuem a maior carga total, com 4 créditos de diferença.

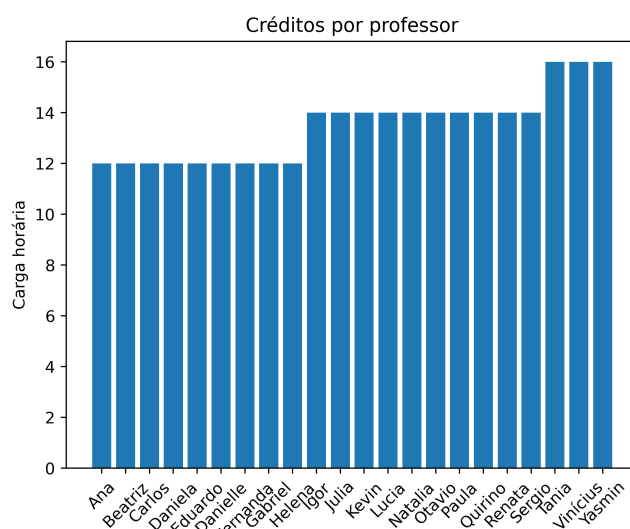


Figura 3: Gráfico da carga horária total de cada professor.

A Figura 4 ilustra a distribuição das disciplinas de acordo com a aptidão dos professores. Esse resultado também é considerado satisfatório, uma vez que um número expressivo de docentes foi alocado em disciplinas com aptidão máxima (nível 5), e nenhum professor foi designado para disciplinas com aptidão mínima (nível -1).

A Figura 5 mostra os custos atribuição e distribuição conforme as iterações são executadas.

Com o objetivo de avaliar a qualidade do programa, foram utilizados os dados da alocação real de disciplinas do 1º semestre de 2025 para a comparação com uma execução do programa. A Figura 6 compara como a carga horária total foi distribuída realmente entre os professores e como uma execução do programa iria distribuir essas mesmas disciplinas. A Figura 7 compara como foi feita a alocação real de disciplinas por aptidão de cada professor e como o programa faria essa alocação, utilizando a mesma planilha de aptidões.

Os resultados do programa foram apresentados ao coordenador responsável pela montagem da grade dos cursos de TADS e BSI. Um dos pontos levantados pelo coordenador foi que se o professor ministrar duas disciplinas iguais, isso deve ter um custo mais baixo do que ministrar duas disciplinas diferentes. Por exemplo, se professora Ana ministrasse duas vezes Programação I seria melhor que se ela ministrasse Programação I e II.

Além disso, foi observado pelo coordenador que cargas horárias de valores iguais podem ter pesos diferentes dependendo de quantas disciplinas são atribuídas. Uma carga horária de 8 horas com 2 disciplinas é diferente de uma carga de mesmo valor com 3 disciplinas, por exemplo. Também foi recomendado que o programa se interrompesse automaticamente com base na porcentagem de melhoria do custo, ao invés de manualmente.

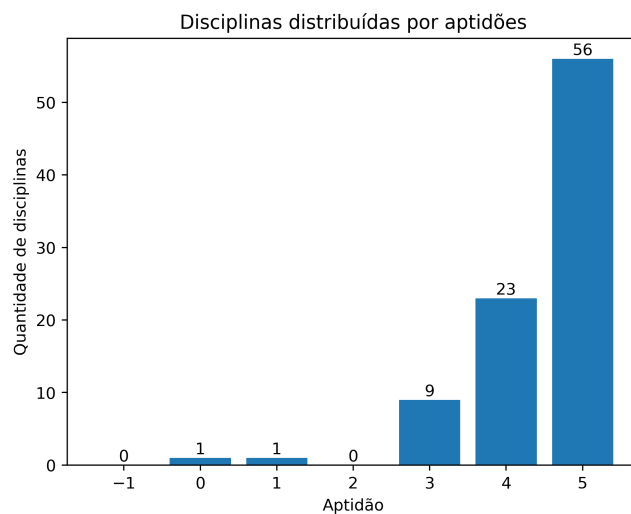


Figura 4: Gráfico das disciplinas alocadas por aptidão dos professores.

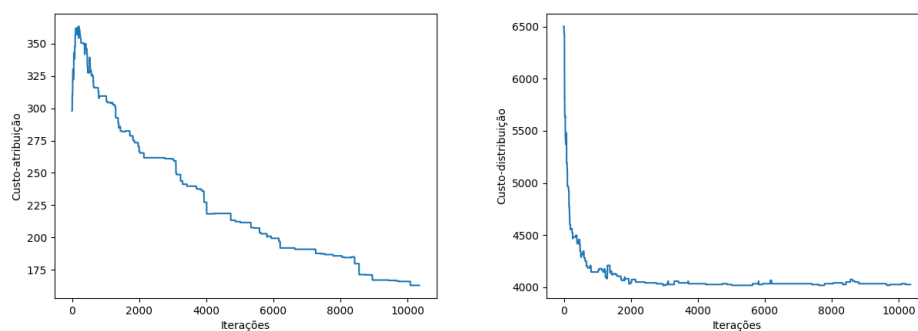


Figura 5: Valor do custo-atribuição (esquerda) e do custo-distribuição (direita) conforme a quantidade de iterações.

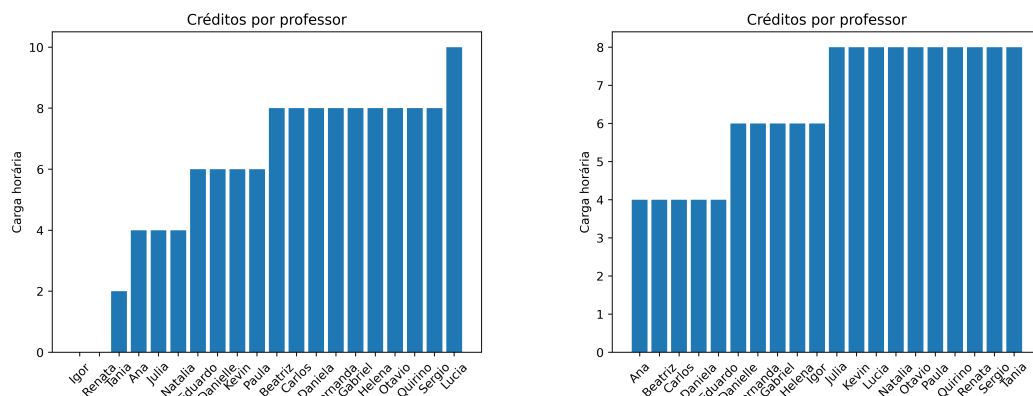


Figura 6: Carga horária total por professor: real (esquerda) e gerada pela alocação automatizada (direita).

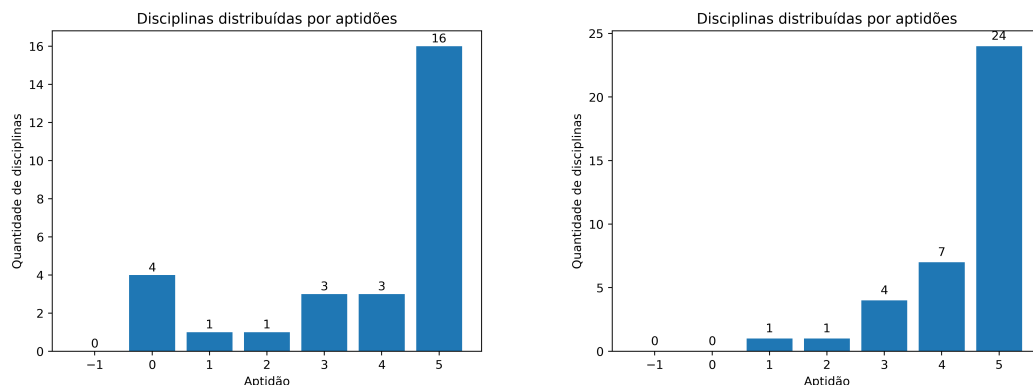


Figura 7: Gráfico da alocação de disciplinas do 1º semestre de 2025 por aptidão dos professores: real (esquerda) e feita pelo programa (direita).

O coordenador ainda relatou outros problemas que excedem a atribuição de disciplinas para professores, como a montagem da grade horária, demanda de alunos e distribuição entre as salas de aula. Há várias restrições que devem ser atendidas, como preferências pessoais dos professores para certos horários, preferência para ministrar disciplinas no mesmo dia ou atender o regulamento que afirma que um professor que ministra aula à noite, não poderá ministrar aula de manhã no dia seguinte. Também é necessário observar que não sejam alocadas disciplinas em um mesmo horário para um professor, assim como uma sala que não seja ocupada por mais de uma disciplina no mesmo horário. Todas estas considerações do coordenador são apontadas neste estudo melhorias a serem realizadas em trabalhos futuros.

5. Conclusões

Este trabalho apresentou uma abordagem baseada em busca local para a otimização da atribuição de disciplinas a professores, considerando dois critérios principais: a aptidão docente e o balanceamento da carga horária. A solução proposta demonstrou resultados promissores, superando a alocação manual tradicional utilizada atualmente, tanto em termos de distribuição equilibrada quanto na adequação dos docentes às disciplinas.

A aplicação prática da metodologia, com base em dados reais e *feedback* do coordenador dos cursos, evidenciou que o sistema é capaz de auxiliar na tomada de decisão, embora ainda existam aspectos do problema real que não foram considerados, como preferências de horários, restrições e ocupação de salas. Tais aspectos serão abordados em trabalhos futuros.

6. Agradecimentos

O presente trabalho foi realizado com apoio da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Código de Financiamento 001 e com apoio financeiro da do Programa de Pós-Graduação em Tecnologia (PPGT) da Faculdade de Tecnologia da Unicamp.

Referências

- Dreo, J., Siarry, P., Petrowski, A., e Taillard, E. (2006). *Metaheuristics for Hard Optimization*. Springer Berlin, Germany. ISBN 978-3-540-23022-9.
- Ehrgott, M. (2005). *Multicriteria Optimization*. Springer Berlin, Germany, 2 edition. ISBN 3-540-21398-8.

- Garruth, F. E. e Resendo, L. (2024). MODELO EXATO PARA O PROBLEMA DA GRADE DE HORÁRIO ESCOLAR CONSIDERANDO CUSTO PEDAGÓGICO E PREFERÊNCIAS DOCENTES: REALIDADE DOS INSTITUTOS FEDERAIS. *Anais do LVI Simpósio Brasileiro de Pesquisa Operacional*.
- Gunantara, N. (2018). A review of multi-objective optimization: Methods and its applications. *Cogent Engineering*, p. 16.
- Kampke, E. H. e Müller, T. M. (2022). ANÁLISE DO IMPACTO DA ALEATORIEDADE E MOVIMENTOS NA META-HEURÍSTICA GRASP COM SIMULATED ANNEALING PARA O PROBLEMA DE TABELA-HORÁRIO DO DCOMP-CCENS-UFES. *Anais do LVI Simpósio Brasileiro de Pesquisa Operacional*.
- Kochenderfer, M. J. e Wheeler, T. A. (2019). *Algorithms for Optimization*. MIT Press, Cambridge, Massachusetts and London, England, 1 edition. ISBN 0262039427.
- Lewis, R. (2008). A survey of metaheuristic-based techniques for university timetabling problems. *OR Spectrum*, 30:167–190.
- Luke, S. (2013). *Essentials of Metaheuristics*. Lulu, 2 edition. ISBN 978-1-300-54962-8.
- Mariotti, T. e Filho, M. A. F. B. (2023). Elaboração de quadros de horários de aulas para o Instituto Federal Goiano: uma abordagem heurística. *Anais do LVI Simpósio Brasileiro de Pesquisa Operacional*.
- MirHassani, S. (2006). A computational approach to enhancing course timetabling with integer programming. *Applied Mathematics and Computation*, 175(1):814–822. ISSN 0096-3003. URL <https://www.sciencedirect.com/science/article/pii/S0096300305006466>.
- Oliveira, T. F., Real, L. B., e Mapa, S. M. S. (2024). OTIMIZAÇÃO DA GRADE DE HORÁRIOS: UM ESTUDO DE CASO NO INSTITUTO FEDERAL DE MINAS GERAIS. *Anais do LVI Simpósio Brasileiro de Pesquisa Operacional*.
- Song, T., Liu, S., Tang, X., Peng, X., e Chen, M. (2018). An iterated local search algorithm for the university course timetabling problem. *Applied Soft Computing*, 68:597–608. ISSN 1568-4946. URL <https://www.sciencedirect.com/science/article/pii/S1568494618302254>.
- Talbi, E.-G. (2009). *Metaheuristics : from design to implementation*. John Wiley & Sons, Inc., Hoboken, New Jersey, 1 edition. ISBN 978-0-470-27858-1.